

E-Commerce Order Management System

Detailed Project Documentation

Spring Boot • Java • JPA • Hibernate • MySQL • JWT • Swagger • Actuator • AOP • Email

A comprehensive description of the implemented backend architecture, entities, security model, business workflows, APIs, persistence, scheduled jobs, notifications, and operational features.

1. Project Overview

The E-Commerce Order Management System is a Spring Boot REST API that implements the core backend operations of an online shopping platform. The system covers catalog management, customers, carts, checkout, orders, payments, reviews, administration, notifications, scheduled processing, API documentation, logging, auditing, and versioning.

The application follows a layered architecture so HTTP handling, business rules, database access, DTO conversion, security, and cross-cutting concerns remain separated.

1.1 Technology Stack

Technology	Purpose
Java	Primary programming language
Spring Boot 4.1.1	Application framework
Spring Web MVC	REST APIs and HTTP handling
Spring Data JPA	Repository and persistence abstraction
Hibernate ORM	JPA implementation
MySQL 8.0.43	Relational database
Spring Security	Authentication and authorization
JWT 0.12.6	JWT token processing
MapStruct 1.6.3	DTO/entity mapping
Lombok	Boilerplate reduction
Spring AOP	Controller execution logging
Spring Mail	Email notifications
Spring Scheduling	Background scheduled jobs
Springdoc OpenAPI	Swagger/OpenAPI documentation
Spring Boot Actuator	Health and metrics

1.2 Project Configuration

Item	Configured value
Artifact	ecommerce-order-management
Group ID	com.example
Base package	com.example.ecommerce
Database	ecommerce_db
API style	REST
Authentication	JWT Bearer token
Persistence	JPA/Hibernate
Development IDE	Eclipse / Spring Tool Suite

2. Architecture

The main request flow is:

Client → Controller → Request DTO → Service → Repository → Entity/JPA → MySQL

The response flow is:

MySQL → Entity/JPA → Repository → Service → Mapper → Response DTO → Controller → Client

2.1 Package Structure

```
com.example.ecommerce ■■■ controller ■■■ service ■ ■■■ impl ■■■ repository ■■■ entity ■■■ dto ■ ■■■  
request ■ ■■■ response ■■■ mapper ■■■ exception ■■■ config ■■■ security ■■■ scheduler ■■■ enums
```

2.2 Responsibilities

- Controllers handle HTTP endpoints, request bodies, path variables, query parameters and multipart uploads.
- Services contain business rules and transactional workflows.
- Repositories communicate with the database using Spring Data JPA.
- Entities represent persistent domain objects and relationships.
- DTOs define API request and response contracts.
- MapStruct converts between DTOs and entities.
- Security components authenticate requests and process JWTs.
- Schedulers execute background operations independently of HTTP requests.

3. Domain Model

Entity	Role in the system
User	Authentication identity, password and role
Customer	Customer profile and ownership root
Category	Groups products into categories
Product	Catalog item with price and stock
Cart	Customer's active shopping cart
CartItem	Product and quantity in a cart
Order	Placed purchase containing order items
OrderItem	Product, quantity and price inside an order
Payment	Payment record associated with an order
Review	Customer rating and comment for a product

3.1 Relationships

Relationship	Type
Customer ↔ User	One-to-one
Product → Category	Many-to-one
Cart → Customer	Customer-owned

Relationship	Type
CartItem → Cart	Many-to-one
CartItem → Product	Many-to-one
Order → Customer	Many-to-one
Order → Payment	One-to-one
Order → OrderItem	One-to-many
OrderItem → Product	Many-to-one
Review → Product	Many-to-one
Review → Customer	Many-to-one

4. Authentication and Authorization

The application uses Spring Security with stateless JWT authentication. Registration creates a user with a BCrypt-encoded password. Login authenticates credentials and generates a JWT. Protected requests send the token through the Authorization header.

4.1 Roles

Role	Implemented access
USER	Authenticated customer operations, product browsing/search, cart, own orders, payments and cus
ADMIN	Administrative product/category/customer/order/payment operations and dashboard

4.2 JWT Flow

1. Client calls /auth/login.
2. AuthenticationManager validates credentials.
3. JwtService creates the token.
4. Client sends Authorization: Bearer .
5. JwtAuthenticationFilter validates the token.
6. Spring Security establishes authentication.
7. URL rules enforce role restrictions.
8. Service-layer checks enforce ownership for customer resources.

4.3 Password Handling

Passwords are encoded with BCryptPasswordEncoder before storage. The raw password is not required in normal API responses.

5. Product Management

- Create and update products.
- Retrieve products by ID.
- Retrieve products with pagination and sorting.

- Search products by name using case-insensitive matching.
- Update an image URL.
- Upload a real image using multipart/form-data.
- Delete products.
- Bulk-create products through CSV.
- Prevent duplicate product names ignoring case.
- Associate products with categories.
- Track created/updated timestamps and entity version.

5.1 Product Image Upload

The multipart endpoint uploads an image, creates the uploads/products directory when necessary, generates a UUID-based filename, saves the file, and stores an image URL such as /images/products/.

WebConfig maps /images/products/** to the local uploads/products/ directory so the saved image can be requested through HTTP.

6. Category Management

- Create category.
- Get category by ID.
- Get all categories.
- Update category.
- Delete category.
- Prevent deletion when products still reference the category.

7. Customer Management

- Create or update customer.
- Get customer by ID.
- Get all customers.
- Delete customer.
- Validate unique customer email.
- Validate unique customer phone.
- Prevent deletion when orders exist for the customer.
- Associate a customer with a User when required for ownership.

8. Cart Management

- Add a product to the cart.
- Add multiple products.
- Import cart items from CSV.
- View the cart.
- Update item quantity.

- Remove a product.
- Clear the cart.
- Validate product stock.
- Enforce customer ownership for USER operations while allowing ADMIN access.

8.1 Cart CSV

The cart CSV workflow reads product names and quantities, resolves the product records, and uses the same cart business logic for adding the items.

9. Checkout

Checkout converts the current cart into an order.

The workflow is:

Customer validation → Cart lookup → Empty-cart validation → Stock validation → Order creation → Order-item creation → Total calculation → Stock reduction → Cart clearing → Confirmation email

9.1 Checkout Rules

- Checkout cannot proceed with a missing or empty cart.
- Each cart quantity must be positive.
- Requested quantity cannot exceed current product stock.
- Order starts with PLACED status.
- Order item price is taken from the current product price.
- Product stock is reduced after order item creation.
- Order total is calculated from quantity × price.
- Cart items are deleted after successful checkout.
- Order confirmation email is sent to the customer email.

10. Order Management

- Create or update orders.
- Retrieve orders.
- Retrieve an order by ID.
- Paginate orders.
- Search orders by status.
- Add and remove order items.
- Validate order status transitions.
- Cancel eligible orders.
- Automatically cancel qualifying old PLACED orders.
- Restore stock when an order is cancelled.
- Prevent deletion of orders that are not in the permitted deletion state.
- Enforce ownership for customer access.

10.1 Order Status Workflow

Current status	Permitted transition examples
PLACED	PAID or CANCELLED
PAID	CONFIRMED or CANCELLED
CONFIRMED	SHIPPED or CANCELLED
SHIPPED	DELIVERED
DELIVERED	No cancellation
CANCELLED	No further transition

The implementation also permits the same status to remain unchanged and rejects invalid backward or skipped transitions.

10.2 Stock Restoration

When an order enters CANCELLED, the quantities from its order items are added back to the corresponding product stock. This keeps inventory consistent with the cancellation workflow.

11. Payment Management

- Create payment.
- Validate that the order exists.
- Prevent duplicate payments for the same order.
- Validate that payment is allowed only for PLACED orders.
- Use the order total for payment validation.
- Set payment date and payment status.
- Change the order status to PAID after successful payment.
- Send a payment-success email.

12. Reviews

- Create product reviews.
- Validate rating between 1 and 5.
- Require a review comment.
- Retrieve reviews by product.
- Retrieve reviews by customer.
- Delete a review.
- Generate a product review summary.

12.1 Review Summary

Field	Meaning
totalReviews	Number of reviews
averageRating	Average of all review ratings
oneStar	Number of 1-star reviews

Field	Meaning
twoStar	Number of 2-star reviews
threeStar	Number of 3-star reviews
fourStar	Number of 4-star reviews
fiveStar	Number of 5-star reviews

13. Admin Dashboard

The admin dashboard aggregates operational information into AdminDashboardResponseDto.

Metric	Meaning
totalCustomers	Total customer records
totalProducts	Total product records
totalOrders	Total order records
totalPayments	Total payment records
totalRevenue	Sum of order totals excluding cancelled orders
pendingOrders	Orders in active workflow states
deliveredOrders	Orders with DELIVERED status

14. Email Notifications

EmailService abstracts email delivery and EmailServiceImpl uses JavaMailSender with SimpleMailMessage.

Trigger	Notification
Checkout	Order confirmation
Successful payment	Payment success
Low stock	Low Stock Alert to configured administrator

SMTP credentials and private application secrets should remain outside source control.

15. Scheduled Jobs

15.1 Automatic Order Cancellation

The scheduled job identifies old PLACED orders and calls a dedicated service method designed for scheduled execution. It does not depend on an HTTP Authentication object because scheduled jobs run outside the normal web request context.

15.2 Low Stock Alert

The low-stock scheduler checks product quantities and sends an email when quantity is at or below the configured threshold. The administrator recipient is configured through app.admin.email.

The development schedule can be changed to a production-friendly daily cron schedule when the application is deployed.

16. Auditing and Versioning

JPA auditing is enabled at application level. Product records use @CreatedDate and @LastModifiedDate for timestamps and @Version for optimistic locking.

- Created date records when the entity was created.
- Updated date records the latest modification.
- Entity version increments when updates occur.
- Optimistic locking helps detect conflicting updates.

17. Transactions

Transactional boundaries are used for workflows that modify multiple related records. Checkout, payment/order processing, stock changes, and cancellation need consistent database updates.

For example, checkout modifies the order, order items, product stock, and cart. Running these operations inside a transaction allows the database changes to succeed or roll back together when an error occurs.

18. Exception Handling

Exception	Business situation
ProductNotFoundException	Product does not exist
CategoryNotFoundException	Category does not exist
CustomerNotFoundException	Customer does not exist
OrderNotFoundException	Order does not exist
OrderItemNotFoundException	Order item does not exist
PaymentNotFoundException	Payment does not exist
InvalidOrderStatusException	Invalid order transition
InsufficientStockException	Requested stock is unavailable
InvalidPaymentException	Payment violates payment rules
InvalidOrderException	General order/business rule violation
CategoryInUseException	Category is referenced by products
DuplicateEmailException	Customer email already exists
DuplicatePhoneException	Customer phone already exists
DuplicateProductException	Product name already exists
CsvFileException	CSV data/file is invalid

GlobalExceptionHandler converts application exceptions and validation failures into a consistent ErrorResponse containing timestamp, HTTP status and message.

19. DTO and MapStruct Design

The API uses request and response DTOs instead of exposing entities directly. This provides a controlled API contract and prevents unnecessary persistence relationships from leaking into responses.

MapStruct mapper interfaces convert DTOs to entities and entities to DTOs. For relationship IDs such as productId, categoryId and customerId, the service loads the actual entity before persistence.

20. AOP Logging

LoggingAspect uses an @Around advice over controller methods. It records the HTTP method, request URI, controller class, method name, and execution time.

Example:

```
API Request: GET /products/1 → ProductController.getProductById()
API Response: ProductController.getProductById() completed in 26 ms
```

This keeps logging separate from controller business logic.

21. Swagger / OpenAPI

Swagger provides interactive documentation for the REST API and supports bearer authentication through the configured bearerAuth security scheme.

Swagger UI: /swagger-ui.html

OpenAPI specification: /v3/api-docs

For pageable endpoints, page and size can be supplied directly. A typical request uses page=0 and size=10. Sorting should use actual entity property names rather than placeholder values.

22. Actuator

Spring Boot Actuator provides operational endpoints. The current configuration exposes health, info and metrics.

Health endpoint: /actuator/health

The health endpoint is permitted by the security configuration.

23. API Endpoint Groups

Module	Endpoint group	Purpose
Authentication	/auth/**	Register and login
Products	/products/**	Catalog management
Product images	/products/{id}/image/upload	Multipart image upload
Product CSV	/products/upload-csv	Bulk import
Categories	/categories/**	Category operations
Customers	/customers/**	Customer operations
Cart	/carts/**	Cart operations
Orders	/orders/**	Order operations
Order items	/order-items/**	Order-item operations
Payments	/payments/**	Payment operations

Module	Endpoint group	Purpose
Checkout	/checkout/{customerId}	Convert cart to order
Reviews	/reviews/**	Review operations
Admin	/admin/**	Administrative dashboard
Swagger	/swagger-ui.html	Interactive documentation
OpenAPI	/v3/api-docs	API specification
Actuator	/actuator/health	Health monitoring

24. End-to-End Demonstration

Step	Demonstration
1	Register user
2	Login and obtain JWT
3	Authorize Swagger with Bearer token
4	Browse/search products
5	View categories
6	Add product to cart
7	Update/remove cart items
8	Checkout
9	Receive order confirmation email
10	Make payment
11	Receive payment-success email
12	View order/status
13	Create review
14	View review summary
15	Login as ADMIN
16	Open dashboard
17	Manage products/categories
18	Upload product CSV/image
19	Observe low-stock alert

25. Technical Concepts Demonstrated

- Spring Boot configuration and dependency injection
- REST API development

- Spring Data JPA and Hibernate
- MySQL database integration
- Entity relationships
- DTO pattern
- MapStruct
- Bean Validation
- Global exception handling
- Transactions
- JPA auditing
- Optimistic locking and versioning
- Pagination and sorting
- Derived repository queries and JPQL
- JWT authentication
- Role-based authorization
- Ownership-based access control
- Multipart file upload
- CSV processing
- Email integration
- Scheduled jobs
- Spring AOP
- Swagger/OpenAPI
- Actuator

26. Project Display / Portfolio Description

This project can be presented as a full-featured Spring Boot E-Commerce Order Management backend. The implementation demonstrates both standard CRUD development and real business workflows such as inventory management, checkout, payment state changes, cancellation with stock restoration, customer ownership, scheduled processing, email notifications, reviews, administrative reporting, and API observability.

A concise portfolio description can be:

“Developed a Spring Boot REST-based E-Commerce Order Management System using Spring Data JPA, Hibernate, MySQL, JWT authentication, role-based authorization, MapStruct, email notifications, scheduled jobs, AOP logging, Swagger/OpenAPI and Actuator. Implemented product/catalog management, cart, checkout, order lifecycle, payment processing, inventory control, reviews, auditing, versioning and an admin dashboard.”

27. Production Considerations

- Use environment variables or a secret manager for database and SMTP credentials.
- Use persistent/object storage for product images instead of relying only on local application storage.
- Use a production database migration strategy rather than relying on ddl-auto=update.
- Configure scheduler intervals according to business requirements.
- Add production monitoring and alerting.

- Consider rate limiting and additional API hardening.
- Use HTTPS in deployment.
- Restrict sensitive actuator endpoints appropriately.
- Keep uploaded files protected against unsafe file types and names.
- Review email delivery, retry and failure handling for production.

28. Conclusion

The E-Commerce Order Management System now represents a broad Spring Boot backend rather than a simple CRUD application. It combines REST APIs, JPA/Hibernate persistence, authentication and authorization, customer ownership, inventory operations, checkout, payment processing, scheduled jobs, email notifications, reviews, auditing, versioning, file processing, API documentation, logging and administration into one project.

The implemented feature set provides a solid basis for demonstrating Java and Spring Boot backend development in a portfolio or technical discussion.